

人工智能导论 project4

StatBot: 面向 AI + 统计的自主数据分析 Agent

自然语言驱动的分析、代码执行与报告生成系统

陆超羿
金志恒
黄麒锐

项目背景：为什么需要统计分析 Agent?

传统统计分析通常需要多个步骤：

1. 读取数据并理解字段含义
2. 检查变量类型、缺失值和异常值
3. 根据研究问题选择统计方法
4. 编写 **Python / R** 代码执行分析
5. 解释统计检验、回归模型或可视化结果
6. 整理图表、代码和文字，形成分析报告

现有痛点：

1. 普通用户不一定具备统计方法选择能力
2. 手动编程成本高，分析流程容易中断
3. 只让大模型自由生成代码，容易出现方法选择不稳定、代码报错和上下文遗忘
4. 外部模型接口超时，整个分析流程容易失败



用户一句话 -> Agent 自动分析

系统输入：

- 用户上传的数据集，例如： CSV / Excel
- 用户的自然语言问题，
例如：
 - “这个数据有多少行多少列？”
 - “分析数值变量之间的相关性”
 - “对 profit 做 OLS 回归分析”

系统输出：

- 统计分析计划
- 自动生成或内置模板注入的 Python 代码
- Jupyter Kernel 执行结果
- 表格、图像和统计解释
- 可导出的 Notebook
- Markdown 分析报告

项目核心目标：

- 自主规划统计分析流程
- 自动选择统计工具或生成代码
- 调用执行环境完成真实计算
- 维护多轮会话记忆
- 在模型超时保持系统可用

系统总体架构

StatBot 采用分层式 **Agent** 架构，将自然语言交互、会话隔离、统计规划、工具调用、代码执行和结果产物管理整合为一条完整分析链路。

1. 前端交互层

用户通过 **Gradio Web** 页面上传数据、提交自然语言问题、查看执行结果，并下载 **Notebook** 或 **Markdown** 报告。

2. 会话管理层

SessionManager 根据浏览器会话 ID 为每个用户创建独立的 **StatBot** 实例，避免不同会话之间的数据、代码和历史状态串扰。

3. 核心 Agent 编排层

Conversation 是系统中枢，负责接收请求、生成统计分析计划、调用工具路由、更新会话记忆，并整合执行结果。

4. 代码生成与工具执行层

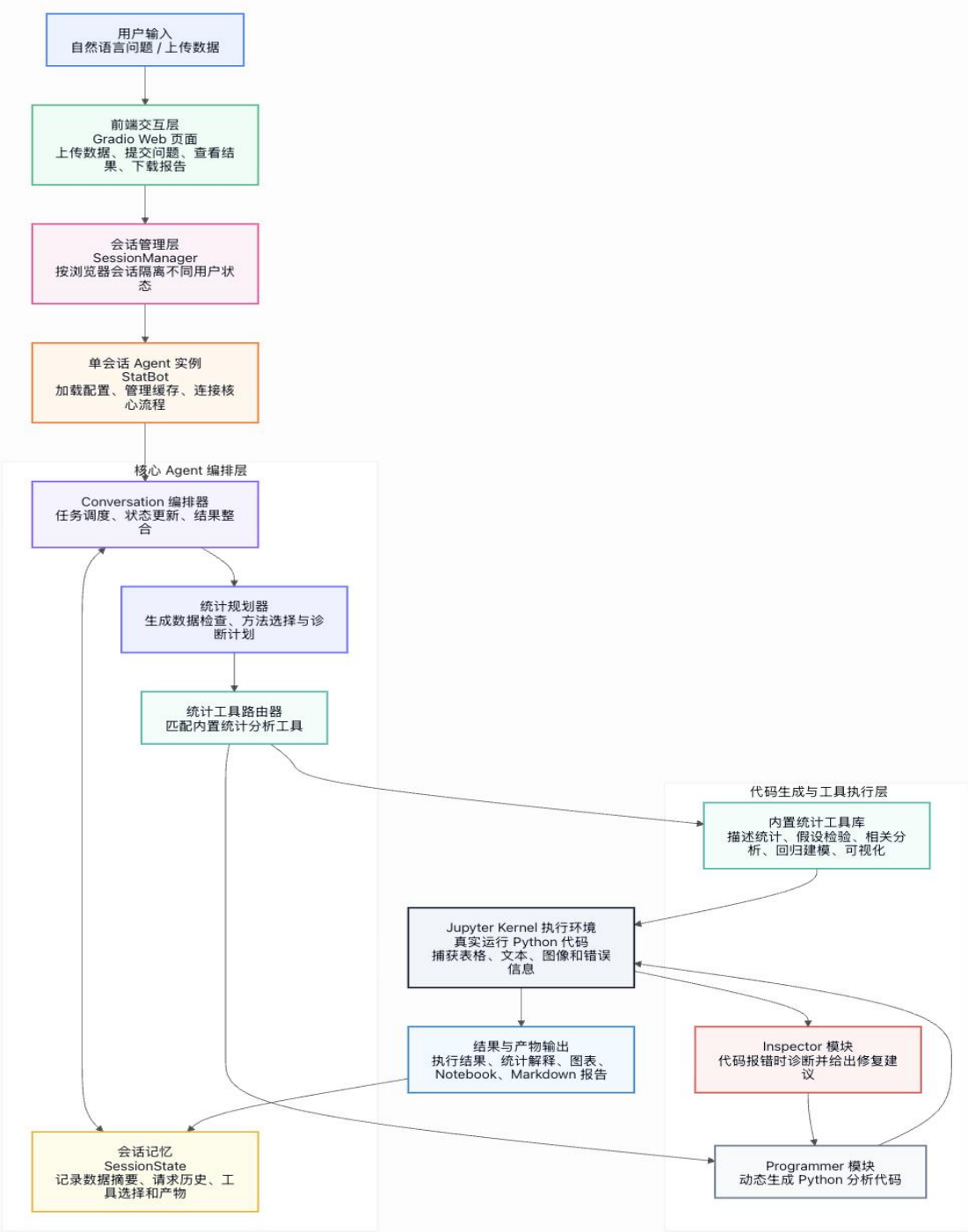
系统优先匹配内置统计工具；若无法匹配，则由 **Programmer** 动态生成 **Python** 代码。代码执行失败时，**Inspector** 会根据错误信息给出修复建议。

5. Jupyter Kernel 执行层

所有分析代码都会在 **Jupyter Kernel** 中真实运行，系统捕获文本、表格、图像和 **traceback**，确保结果来自实际计算而不是模型编造。

6. 结果与产物输出层

系统最终输出统计结果、解释文本、可视化图表、**Notebook** 和 **Markdown** 报告，并将关键产物写入 **SessionState** 供后续分析复用。



StatBot 的一次完整分析流程如下：

1. 用户上传数据集
系统读取文件，并将数据加载为 **Kernel** 中的 **data** 变量。
2. 用户输入自然语言问题
例如：“Run a correlation analysis among numeric variables.”
3. 生成统计分析计划
Agent 先判断任务类型、变量类型、方法选择和诊断要求。
4. 选择执行路径

若命中内置统计工具，则使用结构化代码模板
若未命中，则调用 **Programmer** 模块动态生成代码
5. 执行 **Python** 代码
Jupyter Kernel 返回文本、表格、图像或错误信息。
6. 错误修复
若代码报错，**Inspector** 根据 **traceback** 给出修复建议，**Programmer** 尝试修改代码。
7. 解释结果
系统将执行输出交给模型解释，并给出后续建议。
8. 保存产物
自动记录代码、执行摘要、图片、**Notebook** 和报告。

StatBot 在生成代码前，会先进行统计分析规划，避免直接写代码导致方法选择随意或遗漏关键诊断步骤。

统计规划包含六个部分：

1. 数据概览检查

检查数据行列数、字段类型、缺失值、重复行和基础统计量，确认数据是否适合进一步分析。

2. 研究问题与任务类型识别

判断用户需求属于描述统计、分组比较、相关分析、回归建模、分类预测、时间趋势分析还是可视化任务。

3. 统计方法选择

根据变量类型、样本量、分组数量和研究目标，选择合适的统计检验、相关分析或回归模型。

4. 假设与诊断检查

在分析前或分析过程中检查正态性、方差齐性、异常值、多重共线性、残差分布等统计假设。

5. 推断与建模输出

输出统计量、**p** 值、置信区间、效应量、模型系数、拟合优度和诊断指标。

6. 结果解释与后续建议

用自然语言解释统计结果，并给出下一步分析方向，例如稳健性检验、替代模型或进一步可视化。

核心设计二：内置统计工具注册表

已支持的统计能力：

描述统计与数据质量：

- 数据概览
- 描述统计
- 缺失值分析
- 异常值筛查
- 分组统计
- 类别频数统计
- 列联表汇总

假设检验：

- t 检验
- Wilcoxon 符号秩检验
- Mann-Whitney U 检验
- ANOVA
- Kruskal-Wallis 检验
- 卡方检验
- Fisher 精确检验
- 双比例检验

关系分析与建模：

- Pearson / Spearman 相关分析
- OLS 线性回归
- Logistic 回归
- 趋势分析与可视化

项目在 `builtin_skills.py` 中实现了统计工具注册表。

每个工具包含：

- `name`：工具名称
- `category`：工具类别
- `description`：功能说明
- `matcher`：判断用户请求是否匹配该工具
- `builder`：生成可执行 `Python` 代码
- `rationale`：选择该工具的理由
- `assumptions`：对应统计假设

StatBot 使用 Jupyter Kernel 作为代码执行环境。

执行环境的作用：

- 运行 **Agent** 生成的 **Python** 代码
- 保留跨轮变量，例如上传后的 **data**
- 捕获标准输出、表格、图片和错误信息
- 将执行历史写入 **Notebook**
- 支持用户下载 **notebook.ipynb**

执行输出类型：

- 文本输出
例如行列数、统计检验结果、模型摘要。
- 表格输出
例如描述统计表、相关矩阵、**VIF** 表。
- 图像输出
例如热力图、箱线图、残差图、分布图。
- 错误输出
捕获 **traceback**，并交给 **Inspector** 尝试修复。

项目价值：

- 分析结果来自真实代码执行
- 用户可以复查代码和结果
- 分析过程具有更强可复现性

StatBot 使用 `SessionState` 维护会话记忆。

记录内容包括：

- 上传文件列表
- 当前数据集摘要
- 数据行数和列数
- 字段名与缺失值
- 最近用户请求
- 最近统计计划
- 最近选择的统计工具
- 最近成功执行的代码
- 最近执行结果摘要
- 最近错误信息
- 已生成的图像、文件、**Notebook** 和报告

记忆的作用：

- 支持多轮连续分析
- 让 **Agent** 知道当前正在分析哪个数据集
- 避免重复读取数据
- 为报告生成提供结构化信息
- 为模型超时 **fallback** 提供本地依据

会话隔离：

- **SessionManager** 根据 `request.session_hash` 区分不同浏览器会话
- 每个会话拥有独立的 **StatBot** 实例和 **Jupyter Kernel**
- 避免一个用户看到另一个用户的代码或数据

在真实使用中，外部 **LLM API** 可能因为网络、限流或超时而失败。如果分析代码已经执行成功，但最后一步结果解释失败，用户会误以为整个任务失败。

StatBot 的解决方案：

1. 代码执行与结果解释分离
即使模型解释失败，**Jupyter** 执行结果仍然正常展示。
2. 本地 **fallback summary**
当结果解释模型超时，系统基于执行输出生成本地摘要。
3. 本地 **fallback report**
当报告生成模型超时，系统基于 **SessionState** 生成确定性的 **Markdown** 报告。
4. 保留后续操作建议
即使降级，系统仍会提示用户查看结果、重试解释或编辑代码。

示例输出结构：

```
text
```

```
Local fallback summary
```

```
The analysis code executed successfully,  
but the model-based explanation timed out.
```

```
Observed output preview: ...
```

```
Technical note: ...
```

```
Next, you can:
```

```
[1] Review the execution results above.
```

```
[2] Retry the interpretation step.
```

```
[3] Use Edit Code to refine the analysis.
```

自主决策与规划能力

- 自动生成统计分析计划
- 根据用户问题和数据类型选择分析路径

工具调用与环境交互能力

- 调用文件上传与缓存系统
- 调用 **Jupyter Kernel** 执行 **Python** 代码
- 生成图表、**Notebook** 和 **Markdown** 报告

记忆与上下文管理能力

- 使用 **SessionState** 保存数据摘要、请求历史、工具选择和执行结果
- 支持多轮连续分析

多步骤任务执行能力

完成“上传数据 -> 规划 -> 工具选择 -> 执行代码 -> 修复错误 -> 解释结果 -> 导出报告”的完整流程